Project - Submission Sheet

Software Testing - Input Space Partitioning

Names: Harine Balamuralikrishnan
Group: 04

Software Testing - Summer 2024 - June 14, 2025 - Sibylle Schupp / Daniel Rashedi

Task 2 - Input Space Partitioning

a) Explain why the selected methods are suitable for the input domain model approach (1-2 sentences each)

Answer:

The `get_expired_cookies()` method is suitable for input space partitioning because it processes structured string input and produces different outputs depending on cookie format, the presence of the `expires` attribute, and timestamp comparisons. These behaviors allow meaningful partitioning of the input space.

b) Identify the input domain of your methods (2-3 sentences each)

Answer:

The method accepts a string `cookies` that may contain one or more cookie entries, each composed of key-value pairs separated by semicolons. It also takes an optional float `now` representing the current time. The domain includes valid cookies with `expires`, cookies without it, malformed cookies, and multiple cookie combinations.

c) Similar to Homework 02, gather the results of the following sub-tasks:

1) Identify reasonable characteristics of the possible input data
2) Derive sets of equivalence classes from these characteristics (Note: At least one of your partitions must consist of a number of blocks ≥ 3. Also make sure that you have enough characteristics and corresponding blocks to derive 12 distinct tests from them later on.)

Answer:

1) **Characteristics of the input data:**
   - Cookie Format: Whether the cookie is well-formed and includes an `expires` field.
   - Expiry Timing: The comparison of the `expires` value to `now`.
   - Number of Cookies: Whether the string contains one or multiple cookies.
2) **Equivalence Classes:**
   - Cookie Format:
     - CF1: Proper format with `expires`
     - CF2: Missing `expires`
     - CF3: Invalid format (e.g., missing '=' or malformed syntax)
   - Expiry Timing:
     - ET1: Expired (`expires < now`)
     - ET2: Not expired (`expires > now`)
     - ET3: Boundary (`expires == now`)

- Number of Cookies:
  - NC1: Single cookie
  - NC2: Multiple cookies

The Expiry Timing characteristic contains three blocks, and the combination of all classes allows us to derive at least 12 distinct test vectors.

d) Can you identify any non-valid combinations of blocks from your characteristics? (1-2 sentences each)

Answer:

Yes. Some combinations include invalid cookie formats, such as strings that are not key-value pairs or missing delimiters. For example, `"invalidformatcookie"` or `"a=1, b=2"` (without `expires`) are non-valid combinations that produce no expired output.

e) Derive (at least) one representative value for each equivalence class. Which strategy did you use for the value selection?

Answer:

Representative values were selected using realistic examples and edge conditions. For example:

- Expired: `"id=abc; expires=1000000000"`, `now = 1000000001`
- Future: `"id=abc; expires=3000000000"`, `now = 2000000000`
- Boundary: `"id=abc; expires=2000000000"`, `now = 2000000000`
- No `expires`: `"id=abc"`
- Invalid format: `"invalidformatcookie"`
- Multiple cookies: `"a=1; expires=1000000000, b=2; expires=3000000000"`

f) Combine the selected values from the partitions of each input argument to complete test vectors using `hypothesis`. Document only the excerpt of your code responsible for generation of value combinations.

Answer:

I manually combined values from different equivalence classes to generate 12 distinct test vectors. These test cases span cookie format variations, expiry timings, and single/multiple cookie structures. The values were passed into the Hypothesis strategy `sampled_from()` to enable randomized but controlled test execution.

The excerpt of the code responsible for combining the values into test vectors is shown below:

```
@given(
    st.sampled_from([
        # ---- Single Cookies ----
        ("id=abc; expires=1000000000", 1000000001, [{'name': 'id', 'path': '/'}]),  # T1: expired
        ("id=abc; expires=3000000000", 2000000000, []),                             # T2: future
        ("id=abc; expires=2000000000", 2000000000, [{'name': 'id', 'path': '/'}]),  # T3: boundary
        ("id=abc", 2000000000, []),                                                 # T4: no expires
        ("invalidformatcookie", 2000000000, []),                                    # T5: invalid format

        # ---- Multiple Cookies ----
        ("a=1; expires=1000000000, b=2; expires=3000000000", 2000000000, [{'name': 'a', 'path': '/'}]),  # T6: one expired
        ("a=1; expires=3000000000, b=2; expires=4000000000", 2000000000, []),                             # T7: both future
        ("a=1; expires=2000000000, b=2; expires=2000000000", 2000000000, [{'name': 'a', 'path': '/'}, {'name': 'b', 'path': '/'}]),  # T8: both boundary
        ("a=1, b=2", 2000000000, []),                                                                     # T9: no expires, multiple
        ("randomgarbage, moregarbage", 2000000000, []),                                                   # T10: malformed multiple

        # ---- Mixed Scenarios ----
        ("a=1; expires=1000000000, b=2", 2000000000, [{'name': 'a', 'path': '/'}]),                       # T11: one with and one without expires
        ("a=1; expires=1000000000, b=2; expires=2000000000", 2000000000, [{'name': 'a', 'path': '/'}, {'name': 'b', 'path': '/'}])   # T12: both expired
    ])
)
def test_get_expired_cookies_isp(data):
    cookie_str, now, expected = data
    result = get_expired_cookies(cookie_str, now)
    assert result == expected
```

Each test vector consists of a tuple `(cookie_str, now, expected_output)` and covers a unique combination of the identified input partitions.

g) Create Python tests for your method, which trigger test runs with the selected test vectors / blocks, and assert each individual output. For the submission, merge all these tests into one `*.py` file.

Answer:

The complete test implementation is submitted under the folder `Harine Balamuralikrishnan` in the Git repository. This folder contains the Python file `test_get_expired_cookies_isp.py`, which includes both the `get_expired_cookies()` function and a Hypothesis-based test function covering 12 distinct input space partitioning (ISP) test vectors.

Each test vector is composed of a specific combination of input values (`cookie_str`, `now`, and expected output), and each test asserts the correct behavior of the function. A Markdown file `README.md` is also included in the same folder to document the testing strategy and setup.